TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN HỌC PHẦN

NHẬP MÔN DỮ LIỆU LỚN

XÂY DỰNG HỆ THỐNG THU THẬP VÀ LƯU TRỮ DỮ LIỆU GIAO DỊCH TÀI CHÍNH VỚI KAFKA VÀ MINIO

Lớp : DCT123C4 Giảng viên: Vũ Ngọc Thanh Sang

Võ Huỳnh Minh Sang :	3123411256
Nguyễn Hoàng Long :	3123411179
Châu Quốc Bảo :	3123411024
Nguyễn Đình Cường :	3123411043

TP. Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

BẢNG PHÂN CÔNG CÔNG VIỆC 1
TÓM TẮT 2
CHƯƠNG 1. GIỚI THIỆU 3
1.1. Bối cảnh và bài toán 3
1.2. Mục tiêu và câu hỏi kỹ thuật 3
1.3. Đặc trưng dữ liệu lớn 3
CHƯƠNG 2. CƠ SỞ VÀ CÔNG NGHỆ LIÊN QUAN 4
2.1. Kafka và tiếp nhận dữ liệu 4
2.2. Xử lý luồng và microbatch 4
2.3. Data Lake, MinIO và Parquet 4
2.4. Truy vấn và bảo trì 4
CHƯƠNG 3. DỮ LIỆU VÀ YÊU CẦU 5
3.1. Nguồn dữ liệu 5
3.2. Cấu trúc dữ liệu đầu vào 5
3.3. Đầu ra và hợp đồng bàn giao 7
CHƯƠNG 4. THIẾT KẾ HỆ THỐNG 10
4.1. Kiến trúc và luồng dữ liệu 10
4.2. Xử lý chất lượng và trùng lặp 11
4.3. Offset và giới hạn nhất quán 11
4.4. Cơ sở lựa chọn công nghệ 11
4.5. Lựa chọn phân vùng và hợp đồng bàn giao 12
CHƯƠNG 5. CÀI ĐẶT VÀ TRIỂN KHAI 13
5.1. Thành phần và cấu hình 13
5.2. Trình tự chạy 13

5.3. Phân loại lỗi và hành vi khi chạy lại 13
5.4. Điều kiện tái lập và ghi nhận môi trường 14
CHƯƠNG 6. THỰC NGHIỆM VÀ ĐÁNH GIÁ 15
6.1. Phạm vi số liệu 15
6.2. Thông lượng xử lý trong bộ nhớ 16
6.3. Định dạng lưu trữ và truy vấn 16
6.4. Kiểm tra chất lượng dữ liệu 17
6.5. Kiểm chứng lưu trữ độc lập 17
6.6. Tổng hợp đánh giá và giới hạn phép đo 19
CHƯƠNG 7. KẾT QUẢ DEMO 21
7.1. Demo toàn trình ngày 12/11/2026 24
7.2. Trình tự đọc và kiểm chứng minh chứng 26
CHƯƠNG 8. THẢO LUẬN 27
8.1. Đánh giá đặc trưng Big Data và khả năng mở rộng 27
8.2. Lập luận về tải và kế hoạch kiểm chứng mở rộng 28
8.3. Thứ tự cải tiến và bài học kỹ thuật 29
CHƯƠNG 9. KẾT LUẬN 31
TÀI LIỆU THAM KHẢO 31
PHỤ LỤC A. HƯỚNG DẪN CHẠY VÀ HỒ SƠ BÀN GIAO 33
A.1. Nguồn mã và điều kiện chuẩn bị 33
A.2. Các lệnh chạy theo cấu hình dự án 33
A.3. Cấu hình, schema và minh chứng 34
A.4. Tái lập demo toàn trình bằng môi trường riêng 35
PHỤ LỤC B. ĐỐI CHIẾU YÊU CẦU VÀ DANH MỤC MINH CHỨNG 38
B.1. Vị trí đáp ứng yêu cầu bắt buộc 38

B.2. Kiểm tra tính đầy đủ của hồ sơ.....38
PHỤ LỤC C. SỬ DỤNG CÔNG CỤ AI 39

BẢNG PHÂN CÔNG CÔNG VIỆC

Bảng phân công công việc của các thành viên trong nhóm.

Thành viên	MSSV	Hạ tầng và nguồn dữ liệu	Tiếp nhận và xử lý luồng	Lưu trữ và đối soát	Thực nghiệm, demo và báo cáo
Võ Huỳnh Minh Sang	3123411256	Phụ trách chính: cấu hình Docker Compose, Kafka, topic và Producer sinh giao dịch.	Phối hợp kiểm tra luồng dữ liệu đầu vào.	—	Cung cấp nội dung về hạ tầng, nguồn dữ liệu và Producer.
Nguyễn Hoàng Long	3123411179	Phối hợp kết nối Kafka.	Phụ trách chính: xây dựng ingestion sink, kiểm tra dữ liệu, phân loại lỗi và khử trùng lặp.	Phối hợp kết nối sink với MinIO.	Cung cấp nội dung về Kafka, ingestion và xử lý dữ liệu.
Châu Quốc Bảo	3123411024	—	Phối hợp xác định cấu trúc dữ liệu đầu ra.	Phụ trách chính: MinIO, Parquet, phân vùng, compaction và truy vấn đối soát bằng DuckDB.	Cung cấp nội dung về thiết kế lưu trữ và kết quả kiểm chứng Storage.
Nguyễn Đình Cường	3123411043	—	—	Phối hợp kiểm tra kết quả đầu ra.	Phụ trách chính: benchmark, tổng hợp kết quả demo, phân tích, thảo luận và biên soạn báo cáo nhóm.

TÓM TẮT

Đề tài khảo sát và xây dựng một quy trình Ingestion → Storage cho dữ liệu giao dịch tài chính mô phỏng. Nguồn phát tạo bản ghi giao dịch và gửi vào Apache Kafka; thành phần tiếp nhận đọc dữ liệu, kiểm tra một số quy tắc chất lượng, sau đó ghi dữ liệu hợp lệ dưới dạng Parquet vào MinIO. Dữ liệu được tổ chức theo ngày và loại giao dịch để thuận tiện cho các bước truy vấn và xử lý tiếp theo. Các công cụ hỗ trợ gồm Python, thư viện S3, DuckDB và một nhánh triển khai PySpark Structured Streaming. Mã nguồn, cấu hình và dữ liệu mẫu được lưu cùng dự án. Bằng chứng đã lưu cho phần lưu trữ cho thấy một bộ dữ liệu thử nghiệm 1.000 dòng vẫn giữ nguyên nội dung sau khi gộp 45 tệp thành 9 tệp; đây là thử nghiệm độc lập với Kafka. Bộ benchmark có các phép đo xử lý trong bộ nhớ, kích thước tệp và truy vấn cục bộ; các số liệu này không đại diện cho độ trễ toàn pipeline. Một lượt demo microbatch riêng đã gửi 3.061 message và đối soát 2.923 giao dịch hợp lệ, 61 bản trùng, 77 bản lỗi. Báo cáo phân biệt demo này với thử nghiệm Storage độc lập; khả năng khôi phục sau sự cố vẫn cần được kiểm chứng.

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và bài toán

Các ứng dụng thanh toán tạo ra những sự kiện giao dịch liên tục. Để các nhóm phân tích sử dụng dữ liệu, hệ thống cần tiếp nhận, kiểm tra và lưu trữ dữ liệu theo cấu trúc có thể đọc lại. Nếu chỉ lưu từng tệp CSV rời rạc, việc theo dõi nguồn gốc, kiểm soát lỗi và chia nhỏ khối lượng xử lý sẽ khó khăn hơn khi dữ liệu tăng.

Nhóm chọn phạm vi Ingestion → Storage. Điểm bắt đầu là thông điệp giao dịch trên Kafka; điểm kết thúc là dữ liệu Parquet được lưu trong MinIO và có thể truy vấn. Producer phát dữ liệu mô phỏng được dùng để đồ án hoạt động độc lập. Phần phân tích nghiệp vụ và dashboard thuộc các giai đoạn tiếp theo, không phải mục tiêu bắt buộc của nhóm.

1.2. Mục tiêu và câu hỏi kỹ thuật

Các mục tiêu gồm: mô tả hợp đồng dữ liệu; thiết lập Kafka và MinIO; tiếp nhận và phân loại dữ liệu; tổ chức tệp theo ngày và loại giao dịch; kiểm tra truy vấn và khảo sát ảnh hưởng của định dạng lưu trữ. Những câu hỏi quan trọng là làm sao nhận biết bản ghi không hợp lệ, lưu dữ liệu để dễ truy xuất, và giới hạn nào xuất hiện khi khôi phục sau sự cố.

1.3. Đặc trưng dữ liệu lớn

Velocity thể hiện ở việc giao dịch được phát thành luồng và được gom xử lý theo mẻ. Volume là động cơ lựa chọn Kafka, Parquet và phân vùng, nhưng khối lượng thử nghiệm hiện có chưa chứng minh khả năng mở rộng lên quy mô sản xuất. Veracity thể hiện qua kiểm tra trường và cách ly dữ liệu lỗi. Chạy trên một máy không chứng minh tính chịu lỗi của cụm nhiều nút.

CHƯƠNG 2. CƠ SỞ VÀ CÔNG NGHỆ LIÊN QUAN

2.1. Kafka và tiếp nhận dữ liệu

Apache Kafka tổ chức sự kiện thành topic, mỗi topic gồm một hoặc nhiều partition. Offset xác định vị trí trong từng partition; consumer group cho phép các consumer phối hợp đọc. Thứ tự chỉ được bảo đảm trong cùng partition. Dự án cấu hình một broker và ba partition cho topic giao dịch, hệ số sao chép bằng một; cấu hình này phù hợp demo nhưng không tạo dự phòng broker [1].

2.2. Xử lý luồng và microbatch

Xử lý microbatch gom thông điệp trong khoảng thời gian hoặc theo kích thước mẻ, sau đó kiểm tra và ghi theo nhóm. Dự án có chế độ microbatch bằng Python và nhánh PySpark Structured Streaming riêng. Không nên mặc định hai chế độ có cùng cách xử lý lỗi, checkpoint hay ngữ nghĩa khôi phục [2].

2.3. Data Lake, MinIO và Parquet

Data Lake lưu dữ liệu ở dạng có thể phục vụ nhiều bước xử lý tiếp theo. MinIO cung cấp giao diện lưu trữ đối tượng tương thích S3; đối tượng có key thay vì cấu trúc thư mục vật lý truyền thống. Parquet là định dạng cột, hỗ trợ nén và đọc chọn cột. Trong dự án, đường dẫn phân vùng theo tx_date và tx_type giúp công cụ truy vấn có thể bỏ qua các phân vùng không cần thiết [3], [4].

2.4. Truy vấn và bảo trì

DuckDB được dùng để đọc Parquet và đối soát số dòng, số giao dịch duy nhất và tổng hợp theo phân vùng. Nhiều tệp nhỏ có thể làm tăng chi phí liệt kê và mở tệp. Compaction gộp các tệp trong cùng phân vùng; cần dừng luồng ghi và bảo đảm không có tác vụ đọc hoặc ghi đồng thời khi thực hiện theo hướng dẫn của dự án [5].

CHƯƠNG 3. DỮ LIỆU VÀ YÊU CẦU

3.1. Nguồn dữ liệu

Nguồn chính cho demo là dữ liệu tổng hợp do producer sinh hoặc dữ liệu từ CSV. Dự án cung cấp data/sample_transactions.csv để minh họa; README cũng hướng dẫn tải PaySim từ Kaggle. PaySim là dữ liệu mô phỏng, không được gọi là giao dịch ngân hàng thực tế. Chưa có bằng chứng trong phần kiểm tra này rằng nhóm đã xử lý toàn bộ bộ dữ liệu PaySim [6].

3.2. Cấu trúc dữ liệu đầu vào

Nhóm trường	Ý nghĩa
transaction_id, event_time, step	Định danh, thời gian sự kiện và bước mô phỏng
tx_type, amount, currency	Loại giao dịch, số tiền và tiền tệ
sender_id, receiver_id	Định danh bên gửi và bên nhận
sender_old_balance, sender_new_balance	Số dư người gửi trước và sau
payment_channel, is_fraud_label	Kênh thanh toán và nhãn mô phỏng

Một số quy tắc gồm kiểm tra trường bắt buộc, số tiền dương và loại giao dịch thuộc danh sách cấu hình. Việc xác định đầy đủ schema và khả năng áp dụng các quy tắc cần đối chiếu theo engine được dùng. Dữ liệu sai được đưa vào vùng cách ly trong luồng được thiết kế; nhánh microbatch có gửi cảnh báo Kafka cho một số lỗi.

Bảng 3.2. Schema đầu vào theo producer và Data Contract dự kiến

Trường	Kiểu ví dụ	Ý nghĩa và kiểm tra trong microbatch
transaction_id	String	Mã giao dịch; phải có và không rỗng.
event_time	String	Thời gian sự kiện; dùng để tạo tx_date; validator chưa kiểm tra định dạng.
step	Integer	Bước mô phỏng; validator chưa kiểm tra.
tx_type	String	PAYMENT, TRANSFER, CASH_OUT, CASH_IN hoặc DEBIT; được kiểm tra.
amount	Number	Số tiền; phải chuyển được sang số và lớn hơn 0.

Trường	Kiểu ví dụ	Ý nghĩa và kiểm tra trong microbatch
currency	String	Mã tiền tệ; validator chưa kiểm tra danh mục.
sender_id	String	Mã người gửi; phải có và không rỗng.
sender_old_balance	Number	Số dư trước; validator chưa kiểm tra.
sender_new_balance	Number	Số dư sau; validator chưa kiểm tra.
receiver_id	String	Mã người nhận; phải có và không rỗng.
payment_channel	String	Kênh giao dịch; validator chưa kiểm tra danh mục.
is_fraud_label	Integer	Nhãn mô phỏng; validator chưa kiểm tra.

Danh sách REQUIRED_FIELDS trong config là hợp đồng dự kiến; hàm validate_transaction chỉ kiểm tra một tập con như bảng trên. Chưa thể khẳng định mọi trường trong hợp đồng đều được kiểm tra khi nhận dữ liệu. Kiểu thực tế của tệp Parquet do PyArrow suy ra từ dữ liệu từng mẻ; kiểu ở đây là cấu trúc dự kiến của producer, không phải ràng buộc cứng cho mọi thông điệp.

Ví dụ đầu vào rút gọn: {"transaction_id":"TX01", "event_time":"2026-11-09T08:00:01.120Z", "step":1, "tx_type":"PAYMENT", "amount":250000.0, "currency":"VND", "sender_id":"C01", "sender_old_balance":5000000.0, "sender_new_balance":4750000.0, "receiver_id":"M01", "payment_channel":"MOBILE_APP", "is_fraud_label":0}.

Ba mức mô tả schema cần được phân biệt. Schema dự kiến là hợp đồng giữa producer và bên nhận, gồm tên trường và kiểu mong muốn tại bảng 3.2. Các quy tắc đang kiểm tra là tập điều kiện thực sự được validate_transaction áp dụng trước khi ghi. Schema đọc được sau demo là kiểu mà DuckDB quan sát từ các tệp Parquet và khóa phân vùng của một lượt chạy cụ thể; nó không tự tạo thành ràng buộc cho mọi lượt ghi.

Kiểm tra bổ sung ngày 12/11/2026 thực thi riêng hàm validate_transaction với chín đầu vào có kiểm soát. Script scripts/check_report_schema.py trích hàm bằng AST từ mã nguồn hiện hành và đọc đúng danh mục VALID_TX_TYPES, không khởi động Kafka hoặc MinIO. Hồ sơ docs/schema_validation_probe.json lưu thời điểm chạy, SHA-256 của tệp nguồn, đầu vào và kết quả từng trường hợp. Chín kết quả khớp bảng

dưới mô tả hành vi đang có; đây không phải kết luận rằng validator đã đáp ứng đầy đủ hợp đồng dữ liệu [8].

Trường hợp	Kết quả validator	Ý nghĩa
Bản ghi cơ sở hợp lệ	Chấp nhận	Mốc đối chiếu
amount = "abc"	Từ chối	INVALID_AMOUNT_NUMBER
amount = "100" dạng chuỗi	Chấp nhận	Chuyển thử sang float; chưa ép kiểu giá trị gốc
Thiếu sender_id	Từ chối	MISSING_ACCOUNT_ID
Thiếu event_time	Chấp nhận	Chưa bảo đảm đủ trường hợp đồng
event_time = "not-a-date"	Chấp nhận	Chưa kiểm tra định dạng thời gian
step = "abc"	Chấp nhận	Chưa kiểm tra kiểu step
currency = "XYZ"	Chấp nhận	Chưa kiểm tra danh mục tiền tệ
amount = NaN trong đối tượng Python	Chấp nhận	Chưa kiểm tra số hữu hạn; không phải thử JSON qua Kafka

Do đó, kết luận được giới hạn ở việc lưu và đọc lại được dữ liệu của demo đã đối soát. Chưa khẳng định mọi bản ghi đầu vào được cưỡng chế đúng hợp đồng. Để tăng mức bảo đảm, cần kiểm tra đủ trường, chuẩn hóa kiểu, xác thực timestamp và số hữu hạn, định nghĩa Arrow schema tường minh, rồi kiểm thử tích hợp dữ liệu sai qua Kafka đến Bronze/Quarantine. Các phép thử trên chưa xác định đầu vào bị bỏ lọt sẽ được ghi thành công hay gây lỗi ở bước xử lý sau.

3.3. Đầu ra và hợp đồng bàn giao

Đầu ra chính là các tệp Parquet Snappy dưới tiền tố bronze/transactions, phân vùng tx_date và tx_type. Các bản ghi lỗi được ghi dưới quarantine/bad_transactions. Nhóm tiếp theo cần được bàn giao endpoint, bucket, tiền tố, schema, quy tắc phân vùng và cơ chế cấp quyền; thông tin truy cập thật không nên đưa vào báo cáo công khai.

Bảng 3.3. Các trường và vị trí được bổ sung tại đầu ra microbatch

Thành phần	Các trường hoặc đường dẫn	Mô tả
Bronze	Các trường giao dịch; ingested_at, kafka_partition, kafka_offset	Dữ liệu hợp lệ, kèm thời gian tiếp nhận và vị trí message.
Phân vùng	tx_date=YYYY-MM-DD/tx_type=LOẠI	tx_date lấy từ chuỗi event_time; tx_type nằm trong đường dẫn.
Quarantine	raw_payload, error_reason, kafka_partition, kafka_offset, ingested_at	Lưu thông điệp lỗi và thông tin truy vết; rejected_date dùng làm phân vùng.
Checkpoint microbatch	latest_checkpoint.json	Lưu thống kê của tiến trình; Kafka lưu offset đã commit. Không mặc định khôi phục tập ID chống trùng.

Ví dụ đường dẫn: bronze/transactions/tx_date=2026-11-09/tx_type=PAYMENT/part-<mã>.snappy.parquet. Giá trị trong dấu < > đại diện cho mã tệp được sinh khi chạy.

Ví dụ đầu ra thực tế và schema quan sát được

Bảng dưới lấy nguyên giá trị dòng đầu trong docs/demo_toan_trinh_sql/sample.csv và kiểu từ schema.csv của demo ngày 12/11/2026. Đây là kết quả SELECT đọc dữ liệu đã lưu, không phải bản ghi giả định. Với hive_partitioning=True, DuckDB phục hồi tx_date và tx_type từ đường dẫn; hai cột phân vùng này không nằm trong phần dữ liệu bên trong tệp Parquet [8].

Trường	**Kiểu DuckDB**	**Giá trị thực tế**
transaction_id	VARCHAR	TXN-20261110-00000008-E5EAED
event_time	VARCHAR	2026-11-10T20:14:19.459Z
step	BIGINT	1
amount	DOUBLE	33910476.93
currency	VARCHAR	VND
sender_id	VARCHAR	C668285316
sender_old_balance	DOUBLE	81579360.68
sender_new_balance	DOUBLE	47668883.75
receiver_id	VARCHAR	M253680830
payment_channel	VARCHAR	MOBILE_APP
is_fraud_label	BIGINT	0

Trường	Kiểu DuckDB	Giá trị thực tế
ingested_at	VARCHAR	2026-11-12T13:56:58.945Z
kafka_partition	BIGINT	0
kafka_offset	BIGINT	2
tx_date	DATE	2026-11-10
tx_type	VARCHAR	PAYMENT

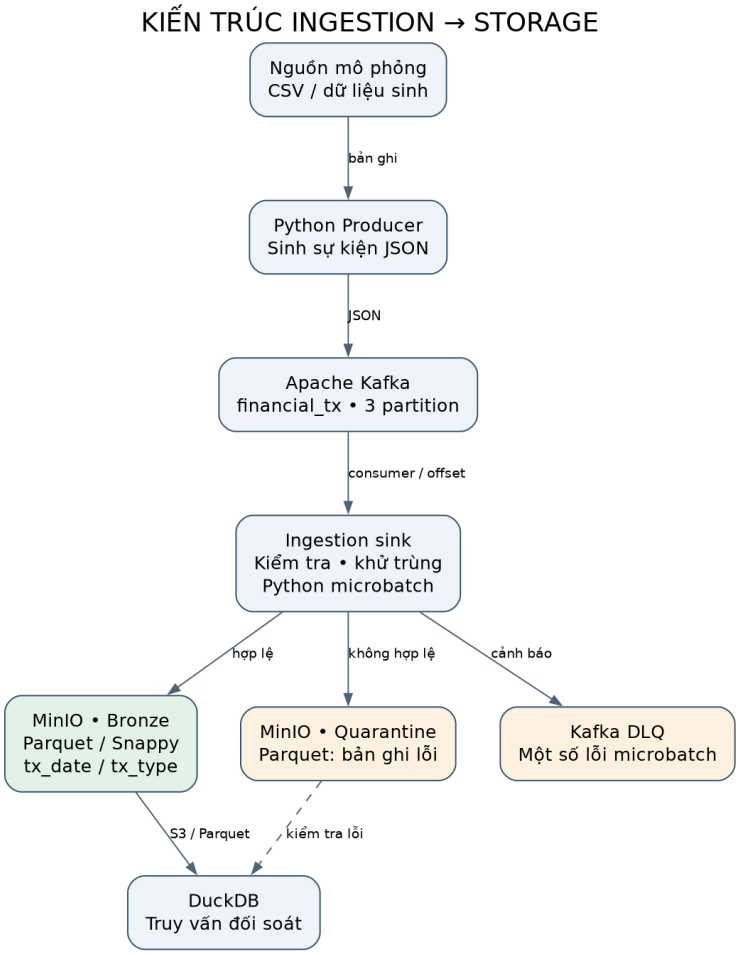
ingested_at ghi thời gian tiếp nhận; kafka_partition = 0 và kafka_offset = 2 cho phép truy vết vị trí message. event_time thuộc ngày 10/11 còn ingested_at thuộc ngày 12/11 thể hiện sự khác nhau giữa thời gian sự kiện mô phỏng và thời gian tiếp nhận. Schema DESCRIBE ghi null = YES cho các cột; điều này không chứng minh dữ liệu có NULL, cũng không phải ràng buộc NOT NULL. amount là DOUBLE trong lượt này; việc đọc được kiểu đó không bảo đảm mọi mẻ tương lai có cùng kiểu.

CHƯƠNG 4. THIẾT KẾ HỆ THỐNG

4.1. Kiến trúc và luồng dữ liệu

Luồng thiết kế: CSV mẫu hoặc dữ liệu tổng hợp → Python Producer → Kafka topic financial_tx → consumer microbatch hoặc PySpark → kiểm tra dữ liệu → MinIO Parquet. Nhánh hợp lệ ghi vào Bronze; nhánh lỗi ghi vào Quarantine. DuckDB đọc dữ liệu đã lưu để đối soát. Với microbatch, một số lỗi còn được gửi tới topic financial_tx_dlq.

Giao diện chính giữa producer và Kafka là message JSON; giữa consumer và MinIO là API tương thích S3; dữ liệu phân tích được đọc ở định dạng Parquet. Các phân vùng Kafka phục vụ tổ chức luồng đọc, còn phân vùng tx_date/tx_type phục vụ tổ chức dữ liệu lưu trữ: hai khái niệm này không đồng nhất.



Hình 4.1. Kiến trúc microbatch và hướng đi của dữ liệu; nhánh PySpark là lựa chọn triển khai riêng.

Sơ đồ mô tả chế độ microbatch đang dùng trong hướng dẫn demo. MinIO lưu Bronze và Quarantine, DuckDB đọc để đối soát; một số thông điệp không hợp lệ được phát thêm vào Kafka DLQ. Với JSON sai cú pháp, nhánh mã hiện tại lưu Quarantine nhưng không phát DLQ.

4.2. Xử lý chất lượng và trùng lặp

Ở microbatch, bản ghi được phân loại trước khi ghi. transaction_id đã gặp được lưu trong một tập trên bộ nhớ của tiến trình để phát hiện trùng. Tập này không được khôi phục sau khi tiến trình khởi động lại, vì vậy không thể suy ra chống trùng xuyên suốt mọi lần chạy. Nhánh PySpark dùng watermark và dropDuplicates; hành vi cần kiểm thử riêng với dữ liệu đến muộn và sự cố.

4.3. Offset và giới hạn nhất quán

Chế độ microbatch ghi dữ liệu và commit offset là những thao tác riêng. Nếu tiến trình dừng sau khi ghi nhưng trước khi commit, việc đọc lại có thể tạo bản ghi trùng. Checkpoint thống kê trên MinIO không thay thế một giao dịch nguyên tử giữa Kafka và object storage. Vì vậy cần kiểm thử sự cố và thiết kế cơ chế ghi lặp an toàn trước khi khẳng định exactly-once.

4.4. Cơ sở lựa chọn công nghệ

Nhóm lựa chọn Kafka để tách bước phát sự kiện khỏi bước ghi Data Lake và sử dụng topic, partition, offset làm giao diện tiếp nhận có thể theo dõi. Nhờ đó producer và sink có thể vận hành với nhịp xử lý khác nhau. So với việc producer ghi thẳng từng giao dịch thành tệp, luồng trung gian cho phép quan sát thông điệp trước khi lưu. Đổi lại, nhóm phải quản lý trạng thái đọc và cửa sổ ghi/commit; cấu hình một broker, replication factor bằng một chưa tạo dự phòng broker [1].

MinIO được chọn làm nơi lưu đối tượng vì phù hợp giao diện S3 mà mã Python và công cụ đối soát đang sử dụng. Môi trường cục bộ giúp nhóm kiểm tra bucket, key và quyền truy cập mà không phụ thuộc tài khoản đám mây. Quyết định này phục vụ sản phẩm độc lập của nhóm; độ bền và khả năng mở rộng nhiều ổ đĩa hoặc nhiều máy chưa được đo trong demo [4].

Parquet Snappy được chọn làm định dạng bàn giao do lưu theo cột và hỗ trợ nén. Phép so sánh cục bộ trên cùng bộ dữ liệu ghi nhận 6,63 MiB so với 34,93 MiB của JSON Lines, giảm khoảng 81%. Số đo hỗ trợ lựa chọn định dạng trong điều kiện phép thử, không đại diện cho tổng dung lượng Kafka, chỉ mục, bản sao và checkpoint của hệ thống [3], [8].

Python microbatch được dùng cho lượt demo vì phù hợp quy mô một máy và cho phép đối soát trực tiếp các nhánh hợp lệ, trùng và lỗi. PySpark là nhánh mã riêng cần thêm môi trường và kiểm chứng tích hợp; không dùng sự hiện diện của nhánh đó làm bằng chứng đã xử lý phân tán. DuckDB phục vụ bước đọc Parquet và kiểm tra bằng SQL, giúp xác nhận đầu ra ngoài thống kê của sink [2], [5].

4.5. Lựa chọn phân vùng và hợp đồng bàn giao

Khóa tx_date phục vụ truy vấn theo thời gian; tx_type chia dữ liệu theo nhóm nghiệp vụ có danh mục hữu hạn. Cách tổ chức này phù hợp truy vấn lọc ngày và loại giao dịch trong đồ án. Nhóm không dùng transaction_id làm khóa đường dẫn vì mỗi ID có thể tạo thêm một vùng lưu rất nhỏ. Kafka partition tổ chức việc đọc message, còn Hive partition tổ chức các tệp trên object storage; số lượng hai loại phân vùng không cần bằng nhau.

Mỗi mẻ tạo tệp mới trong các phân vùng có dữ liệu. Khi có nhiều mẻ nhỏ, số tệp tăng dù số dòng chưa lớn; vì vậy compaction là công việc bảo trì cần thiết của thiết kế. Thử nghiệm Storage đã gộp 45 tệp thành 9 tệp, giảm 80% số tệp và giữ nguyên 1.000 dòng, hash nội dung và các kết quả tổng hợp. Chưa có phép đo chứng minh mức tăng tốc truy vấn sau compaction trên bộ dữ liệu này.

Khi bàn giao, nhóm cung cấp endpoint S3, bucket, tiền tố Bronze và Quarantine, schema đọc được, ý nghĩa từng trường và quy tắc phân vùng. Bên nhận sử dụng hive_partitioning để đọc lại tx_date và tx_type từ đường dẫn. Hồ sơ cũng cần ghi lượt chạy, giới hạn kiểm tra schema và trạng thái writer; quyền truy cập được cấp riêng. Những thông tin đó giúp giai đoạn Storage → Processing đọc đúng tập dữ liệu mà không phải suy đoán từ tên tệp.

CHƯƠNG 5. CÀI ĐẶT VÀ TRIỂN KHAI

5.1. Thành phần và cấu hình

Docker Compose cấu hình Kafka KRaft, Kafka UI, MinIO và các dịch vụ khởi tạo topic, bucket. Mã Python được tổ chức thành producer, ingestion sink, storage management và benchmark. Cấu hình đặt trong src/config.py và biến môi trường. requirements.txt mô tả các thư viện; cần kiểm tra tương thích phiên bản Python, Java, Spark và các gói kết nối trước khi triển khai trên máy mới.

5.2. Trình tự chạy

1. Cài Docker, Docker Compose, Python và các phụ thuộc; cấu hình biến môi trường theo README.

2. Chạy docker compose up -d; kiểm tra broker, topic và bucket đã sẵn sàng.

3. Chạy python -m src.producer.transaction_producer với số lượng và tốc độ mong muốn.

4. Chạy python -m src.ingestion_sink.spark_kafka_to_lake --engine microbatch --batch-size 5000.

5. Dừng luồng ghi trước khi compaction; chạy truy vấn đối soát và lưu log, ảnh, thời gian cùng cấu hình.

Đây là tóm tắt hướng dẫn; tham số đầy đủ và yêu cầu riêng cho PySpark xem README. Việc chạy lại dùng dữ liệu, offset và bucket đã tồn tại có thể tạo kết quả khác. Khi demo cần ghi rõ điều kiện ban đầu và tránh xóa dữ liệu nếu chưa được phép.

5.3. Phân loại lỗi và hành vi khi chạy lại

Trong microbatch, JSON sai cú pháp được lưu vào Quarantine. Một số lỗi nghiệp vụ do validate_transaction phát hiện được lưu Quarantine và phát thêm message DLQ. Vì hai nhánh có điều kiện khác nhau, số dòng Quarantine bằng số message DLQ chỉ được kỳ vọng trong kịch bản lỗi phù hợp, chẳng hạn ba dạng lỗi đã chèn ở demo; không áp dụng đẳng thức này cho mọi đầu vào.

Bản trùng được phát hiện bằng transaction_id trong bộ nhớ tiến trình và không được ghi thêm vào Bronze trong lượt chạy đang xét. Sau khi khởi động lại, tập ID không được phục hồi. Ngoài ra, nếu dữ liệu đã lên MinIO nhưng offset chưa được commit, lần đọc sau có thể nhận lại message. Do đó, checkpoint thống kê không đủ để xác nhận chống trùng qua khởi động lại hoặc exactly-once.

Nếu thiếu event_time, validator hiện có thể chấp nhận bản ghi nhưng bước tạo tx_date truy cập trực tiếp trường này và có nguy cơ lỗi. Nếu amount ở dạng chuỗi số, phép kiểm tra chuyển thử sang float nhưng không thay giá trị gốc. Hai trường hợp minh họa vì sao kiểm tra tại đầu vào và schema lưu trữ phải được thiết kế thống nhất; phép thử hàm riêng trong mục 3.2 chưa chứng minh hành vi ghi ở các bước sau.

5.4. Điều kiện tái lập và ghi nhận môi trường

Lượt chạy tái lập cần lưu cùng lúc cấu hình Compose, mã nguồn, phiên bản phụ thuộc, tên topic/group/bucket và điều kiện dữ liệu ban đầu. Không lấy offset hoặc bucket của lượt cũ làm đầu vào cho một phép so sánh mới nếu chưa ghi rõ. requirements.txt dùng giới hạn phiên bản tối thiểu; khi cài trên máy khác cần lưu pip freeze và digest image cùng log của lượt mới.

Các thông tin máy cần ghi ở lần thực nghiệm tiếp theo gồm hệ điều hành, CPU, số nhân, RAM, ổ đĩa, dung lượng khả dụng và giới hạn tài nguyên Docker. Với benchmark cần ghi cả thư mục chạy và trạng thái cache nếu có kiểm soát. Báo cáo hiện chỉ sử dụng thông số có trong hồ sơ, nên chưa thể tách ảnh hưởng phần cứng khỏi ảnh hưởng kích thước mẻ.

Phụ lục A mô tả lệnh chạy tổng quát và demo cô lập; Phụ lục B tổng hợp điều kiện đọc kết quả và danh mục bàn giao. Nhóm dùng bộ minh chứng đã lưu làm mốc đối chiếu; các thay đổi thời gian, UUID và số tệp do lịch gom mẻ phải được giữ trong log mới thay vì chỉnh cho giống lượt cũ.

CHƯƠNG 6. THỰC NGHIỆM VÀ ĐÁNH GIÁ

6.1. Phạm vi số liệu

Tệp docs/benchmark_results.json ghi nhận một lượt benchmark trên 100.000 giao dịch tổng hợp. Các phép thử khác nhau về phạm vi: phép một xử lý trong bộ nhớ, phép hai so sánh kích thước tệp, phép ba so sánh truy vấn cục bộ và phép bốn phân loại trong bộ nhớ. Không gộp chúng thành một lượt kiểm thử Kafka → MinIO.

Hồ sơ demo toàn trình ngày 12/11/2026 ghi Linux x86_64 và Python 3.14.4; Compose dùng Kafka image apache/kafka:3.7.0 và MinIO image elestio/minio:latest trên một máy. Các phiên bản thư viện được ghi nhận gồm confluent-kafka 2.16.1, PyArrow 26.0.1, DuckDB 2.0.0, pandas 3.1.1 và MinIO Python SDK 7.2.20. CPU, số nhân, RAM và loại ổ đĩa chưa được lưu đủ trong hồ sơ của lần đo; không lấy cấu hình máy hiện tại để gán ngược cho thí nghiệm cũ. Nhãn latest không cố định phiên bản, nên lượt chạy mới cần lưu image ID hoặc digest [8].

Bốn thí nghiệm benchmark được lưu ngày 11/11/2026, tách biệt khỏi demo toàn trình ngày 12/11. Bộ benchmark sinh 100.000 bản ghi mô phỏng; các phép thử xử lý dữ liệu trong tiến trình hoặc đọc/ghi tệp cục bộ. Log TV4 có mô tả phần cứng tổng quát nhưng không đủ cơ sở khẳng định đó là cấu hình chính xác của lần benchmark; do đó không dùng mô tả đó làm thông số máy đã xác minh.

Thước đo được hiểu như sau: throughput = số bản ghi chia cho thời gian đo; latency trong thí nghiệm 1 là thời gian xử lý toàn mẻ, không phải thời gian của một giao dịch đi từ producer tới storage. Dung lượng được tính theo MiB (1 MiB = 1.048.576 byte). Các hình và bảng trong bản báo cáo này đều dùng MiB; số liệu được giữ nguyên từ bộ benchmark gốc.

6.2. Thông lượng xử lý trong bộ nhớ

Kích thước mẻ	Thời gian (ms)	Thông lượng (bản ghi/s)
1.000	12,18	82.120,7
5.000	22,55	221.742,6
10.000	33,29	300.394,8
20.000	67,07	298.191,0

Ở từng mức mẻ, chương trình lấy dữ liệu đầu danh sách, duyệt một lần qua hàm kiểm tra, dùng set nhận diện ID lặp và tạo bảng Arrow từ các dòng hợp lệ. Đồng hồ chỉ bao trùm các bước này, không bao gồm gửi Kafka, nén Parquet thành tệp hoặc tải đối tượng lên MinIO. Mỗi mức trong mã được đo một lượt; chưa có độ lệch chuẩn, khoảng tin cậy hay so sánh trên nhiều lần chạy.

Số đo tăng từ 82.120,7 lên 300.394,8 bản ghi/giây khi tăng mẻ từ 1.000 lên 10.000, tương đương khoảng 3,66 lần trong bộ kết quả này. Mẻ 20.000 đạt 298.191,0 bản ghi/giây, không tăng thêm đáng kể so với mẻ 10.000; thời gian xử lý mẻ tăng từ 33,29 lên 67,07 ms. Đây là quan sát trên phép thử cụ thể, chưa xác định được nguyên nhân bằng profiling.

6.3. Định dạng lưu trữ và truy vấn

Định dạng	Kích thước	So với JSON
JSON Lines	34,93 MiB	Mốc so sánh
CSV	13,90 MiB	Giảm 60,2%
Parquet Snappy	6,63 MiB	Giảm 81,0%

Phép so sánh dung lượng ghi cùng dữ liệu thành JSON Lines, CSV và Parquet Snappy tại thư mục cục bộ, sau đó lấy kích thước từng tệp. Tỷ lệ giảm = (1 − kích thước định dạng so sánh / kích thước JSON) × 100%. Với kết quả đã lưu, Parquet 6,63 MiB so với JSON 34,93 MiB cho mức giảm khoảng 81%; kết quả thay đổi theo dữ liệu, cách biểu diễn và cấu hình nén.

Phép truy vấn ghi một tệp Parquet phẳng và một tập tệp Hive partition theo tx_date/tx_type; cả hai dùng cùng bộ dữ liệu và điều kiện lọc cùng ngày, loại TRANSFER. Mã chạy mỗi truy vấn 10 lần liên tiếp và chia tổng thời gian cho 10. Kết quả 6,33 ms so với 3,33 ms gợi ý phân vùng có ích trong điều kiện chạy này. Chưa có

thử nghiệm độc lập nhiều lần, kiểm soát cache hoặc đo bytes đọc, nên không suy ra mức tiết kiệm I/O cụ thể hay tốc độ tăng ở quy mô khác [5].

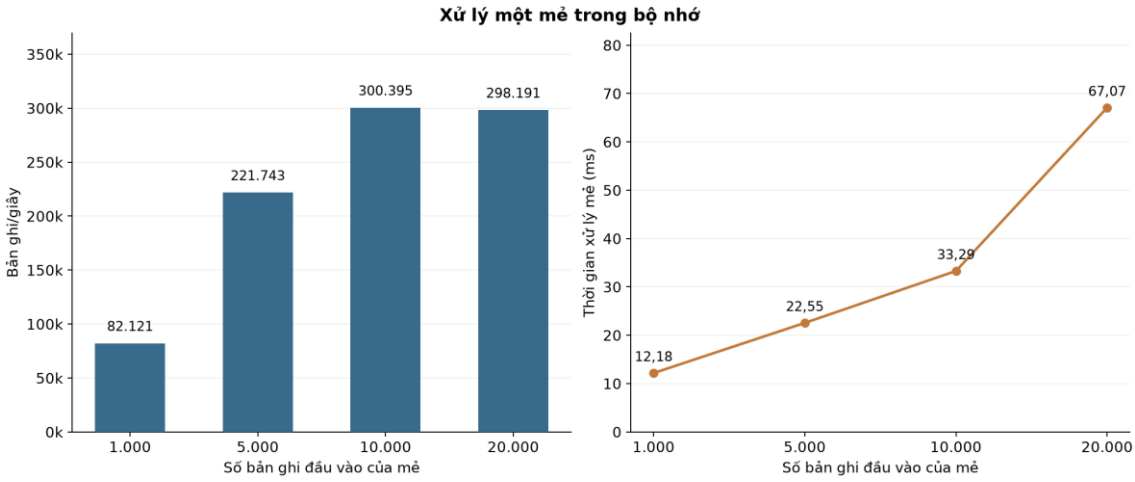
6.4. Kiểm tra chất lượng dữ liệu

Phép kiểm tra chất lượng xây dựng 10.000 bản ghi ban đầu, ghép thêm 500 bản sao và 500 bản ghi đã chèn lỗi. Chương trình lần lượt validate và so ID đã gặp trong một set; số được đếm là số đầu vào thuộc từng nhánh, không phải số bản ghi đã thực sự đi qua Kafka/MinIO. Trên tập thử này kết quả 10.000 hợp lệ, 500 trùng và 500 lỗi phù hợp số lượng dự kiến; không thể suy rộng thành tỷ lệ đúng 100% với mọi dữ liệu hoặc mọi tình huống lỗi.

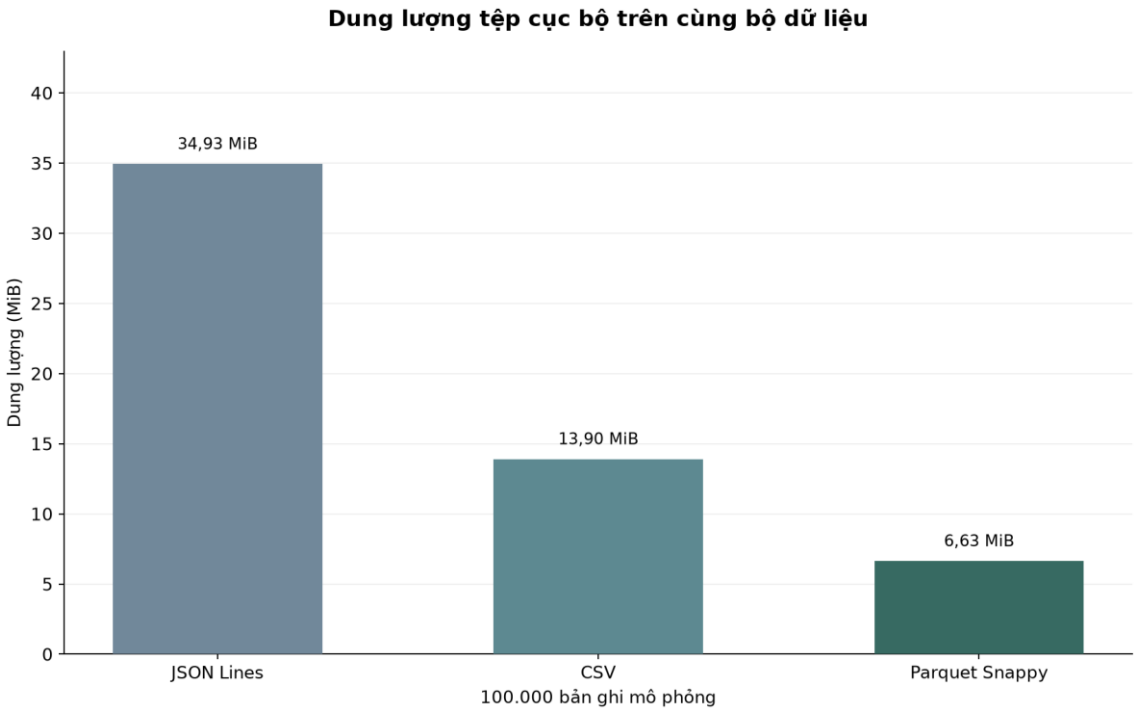
Ví dụ lỗi được chèn là số tiền âm, thiếu sender_id và tx_type không hợp lệ. Kịch bản không bao trùm mọi trường schema, JSON hỏng hoặc lỗi hạ tầng. Phương pháp đánh giá tốt hơn là lưu nhãn mong đợi cho từng đầu vào, đối chiếu nhãn đầu ra và bổ sung các trường hợp dữ liệu biên.

6.5. Kiểm chứng lưu trữ độc lập

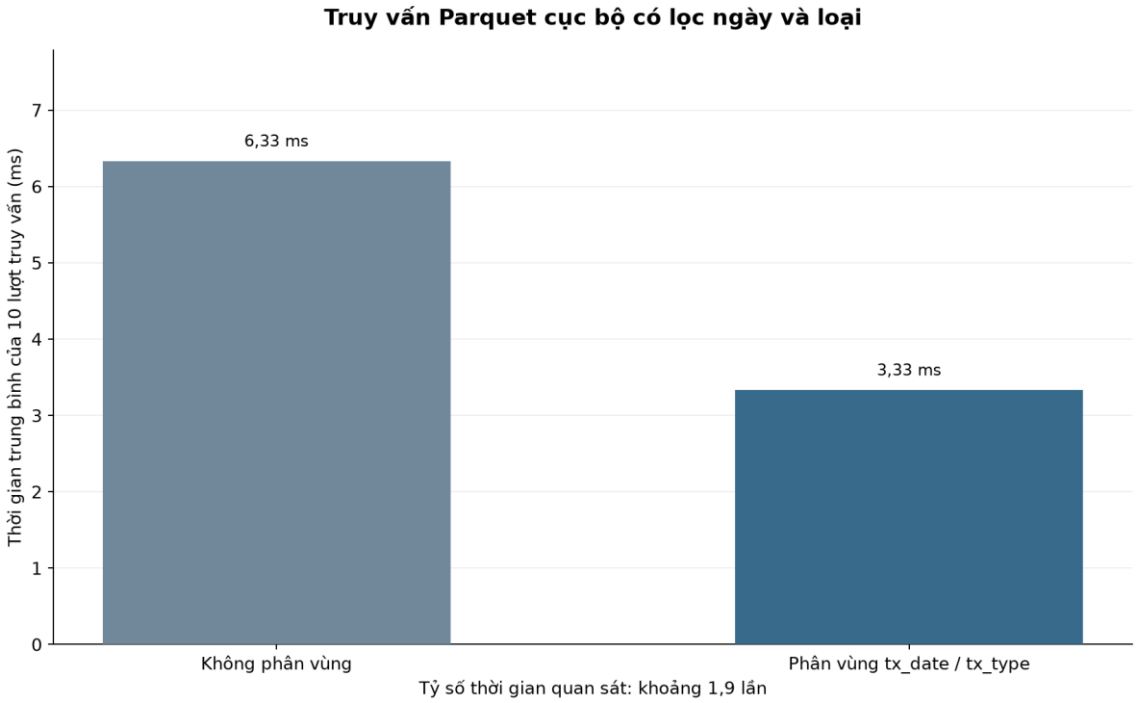
Minh chứng TV3 sử dụng 1.000 giao dịch tổng hợp và sáu bản ghi bị từ chối, không cần Kafka. Kết quả ghi nhận 45 tệp trước và 9 tệp sau compaction; số dòng, hash nội dung đầy đủ, kết quả tổng hợp và dữ liệu cách ly được ghi nhận không thay đổi. Môi trường dùng MinIO chạy cục bộ một ổ đĩa. Đây là bằng chứng kiểm thử Storage, không phải bằng chứng luồng Kafka toàn trình.



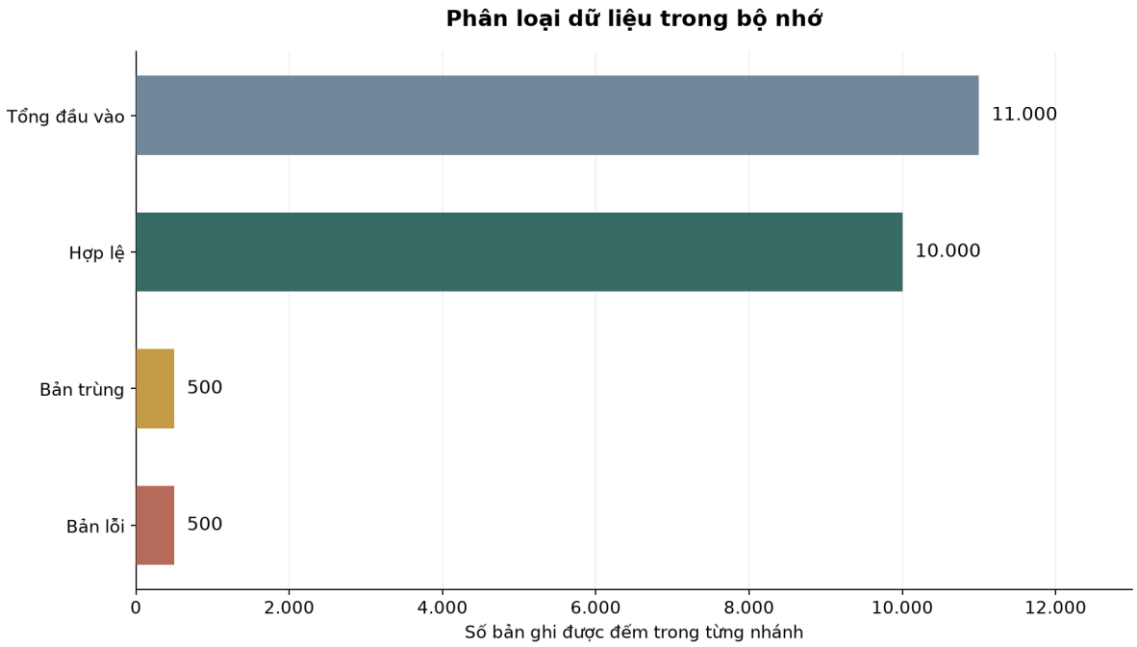
Hình 6.1. Thời gian và thông lượng xử lý trong bộ nhớ; không bao gồm Kafka và MinIO.



Hình 6.2. So sánh kích thước các định dạng trong bộ dữ liệu thử nghiệm.



Hình 6.3. So sánh thời gian truy vấn cục bộ giữa hai cách bố trí tệp.



Hình 6.4. Số bản ghi được phân loại trong bộ nhớ; phép thử không thực hiện ghi Kafka hoặc MinIO.

6.6. Tổng hợp đánh giá và giới hạn phép đo

Thực nghiệm trả lời ba nhóm câu hỏi. Demo Kafka → MinIO kiểm tra các thành phần kết nối và bảo toàn số lượng theo nhánh; benchmark cục bộ khảo sát chi phí xử lý, dung lượng và truy vấn; thử nghiệm Storage kiểm tra nội dung trước/sau bảo trì. Mỗi kết quả chỉ được diễn giải trong phạm vi dữ liệu và phương pháp đã dùng.

Với demo, 2.923/3.061 message, khoảng 95,49%, trở thành dòng Bronze; 61/3.061, khoảng 1,99%, được loại là bản trùng; 77/3.061, khoảng 2,52%, được cách ly. Đây là tỷ lệ trên tổng message thực gửi, có bao gồm bản sao. Tỷ lệ lỗi cấu hình 3% áp dụng cho 3.000 sự kiện gốc; quan sát 77/3.000, khoảng 2,57%, không mâu thuẫn với việc chèn lỗi theo xác suất. Những tỷ lệ này không phải độ chính xác phân loại trên mọi dữ liệu.

Trong benchmark xử lý, mẻ 10.000 cho thông lượng lớn nhất trong bốn cấu hình đã đo, nhưng chưa thể gọi đó là kích thước tối ưu của pipeline. Phép đo chưa chứa thời gian truyền Kafka, nén tệp hoàn chỉnh và tải lên MinIO. Tương tự, mức nhanh hơn khoảng 1,9 lần của truy vấn phân vùng chỉ áp dụng cho truy vấn lọc đã chọn; các truy vấn không lọc ngày/loại có thể cần đọc phạm vi khác.

Tiêu chí đã có bằng chứng gồm: số gửi khớp số tiêu thụ, số hợp lệ khớp Bronze, bản sao khớp số trùng bị loại, lỗi chèn khớp Quarantine và DLQ trong demo, ID trong

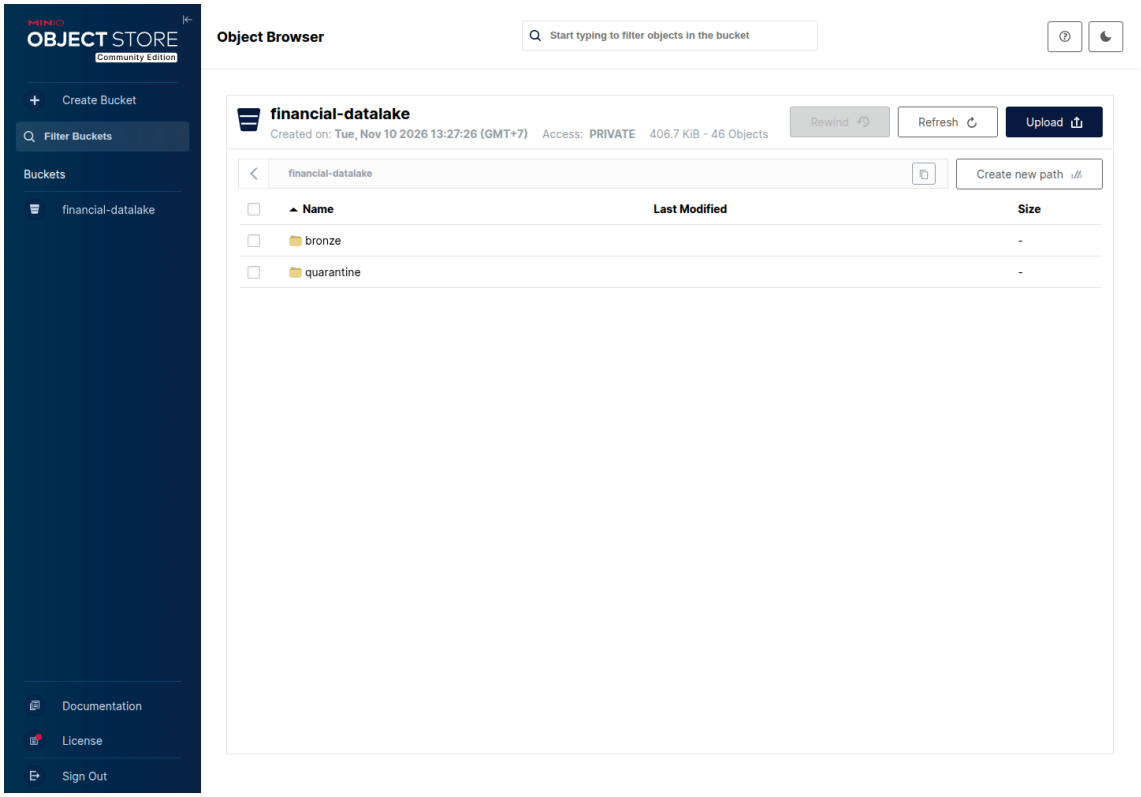
Bronze duy nhất, và compaction giữ nội dung ở thử nghiệm Storage. Tiêu chí chưa được đo gồm: độ trễ p95/p99 toàn trình, mức tải liên tục tối đa, chịu lỗi hạ tầng, đối soát từng payload với nguồn và khôi phục sau sự cố.

Vì vậy, kết quả hiện hỗ trợ kết luận hoàn thành luồng lưu trữ và truy xuất cho kịch bản demo một máy. Việc đáp ứng một hợp đồng schema nghiêm ngặt và vận hành ở tải lớn liên tục cần các phép thử bổ sung trong Chương 8; không chuyển kế hoạch đó thành thành tích đã thực hiện.

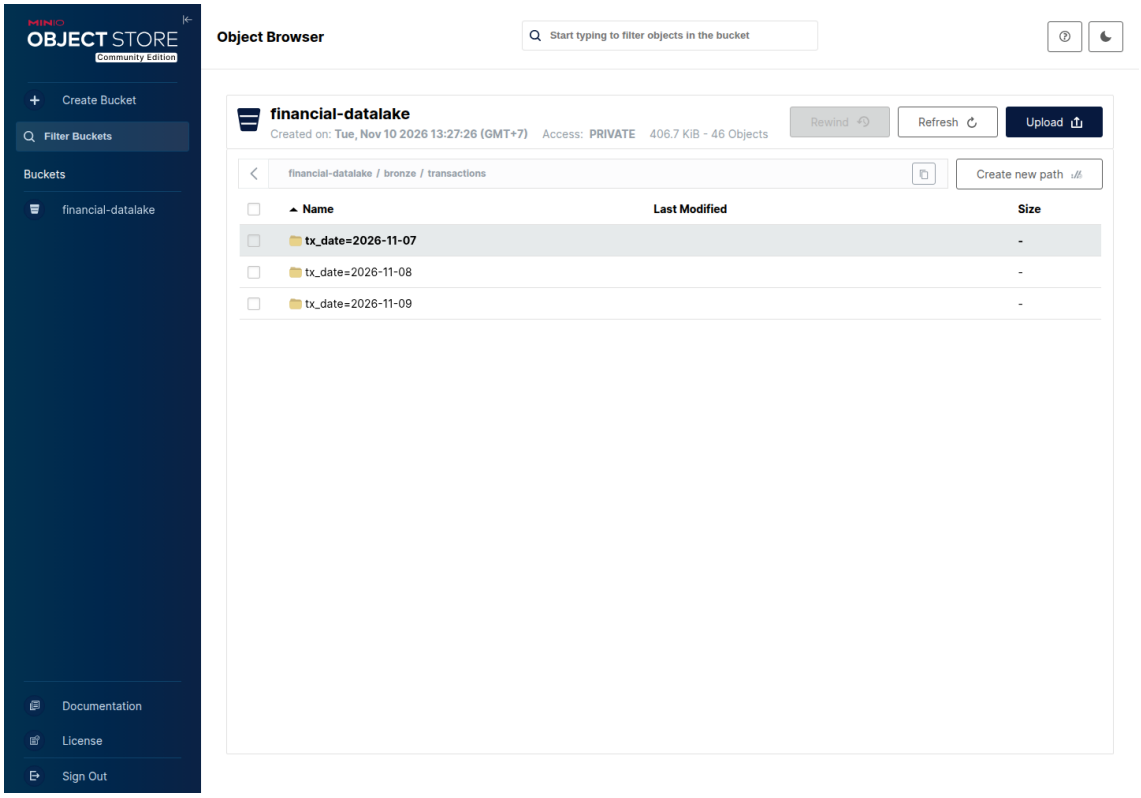
CHƯƠNG 7. KẾT QUẢ DEMO

Các tệp minh chứng Storage gồm docs/tv3_evidence/result.json, dữ liệu đối soát trước và sau, cùng ảnh MinIO trong docs/screenshots. Nhóm có thể trình bày schema, phân vùng, kết quả truy vấn và ảnh trước/sau compaction theo thứ tự trong buổi demo. Cần nêu rõ fixture được tạo riêng cho kiểm chứng Storage.

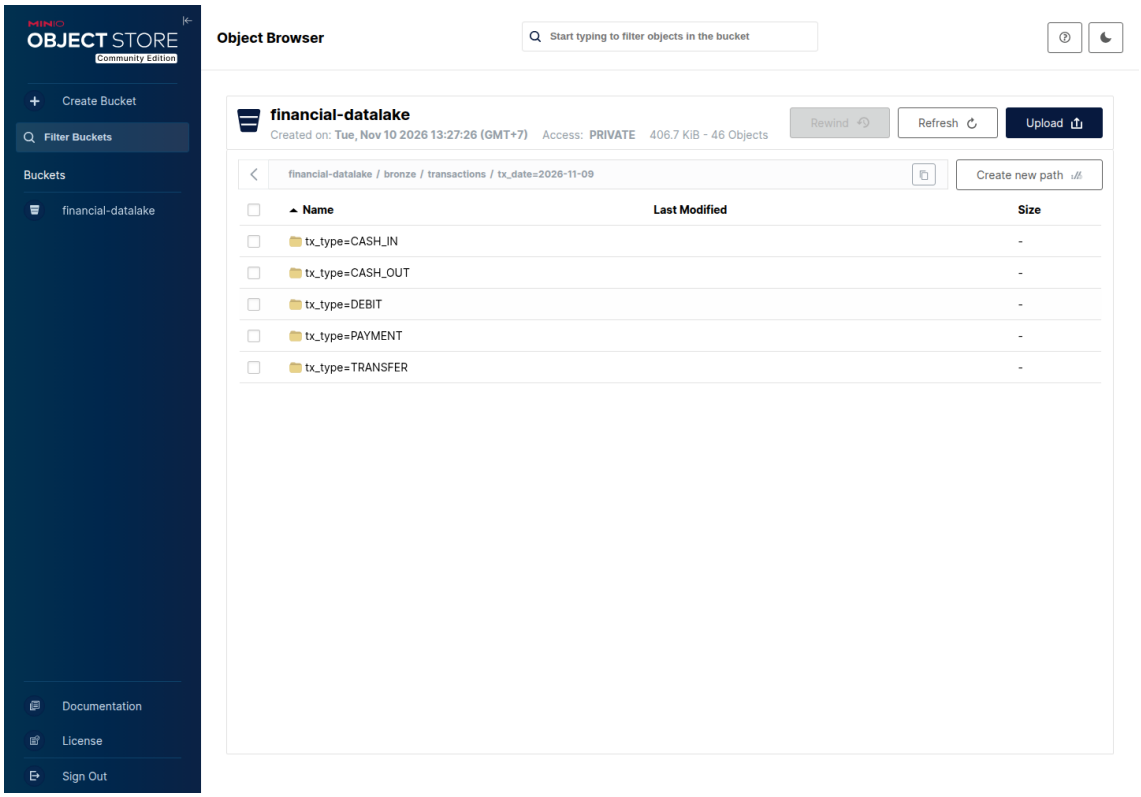
Các biên bản TV1 và TV2 trước đây ghi số liệu 20.200 message nhưng chưa được đối chiếu lại với log gốc. Vì vậy báo cáo sử dụng lượt demo riêng ngày 12/11/2026 bên dưới làm bằng chứng toàn trình; không gộp hai lượt chạy với nhau.



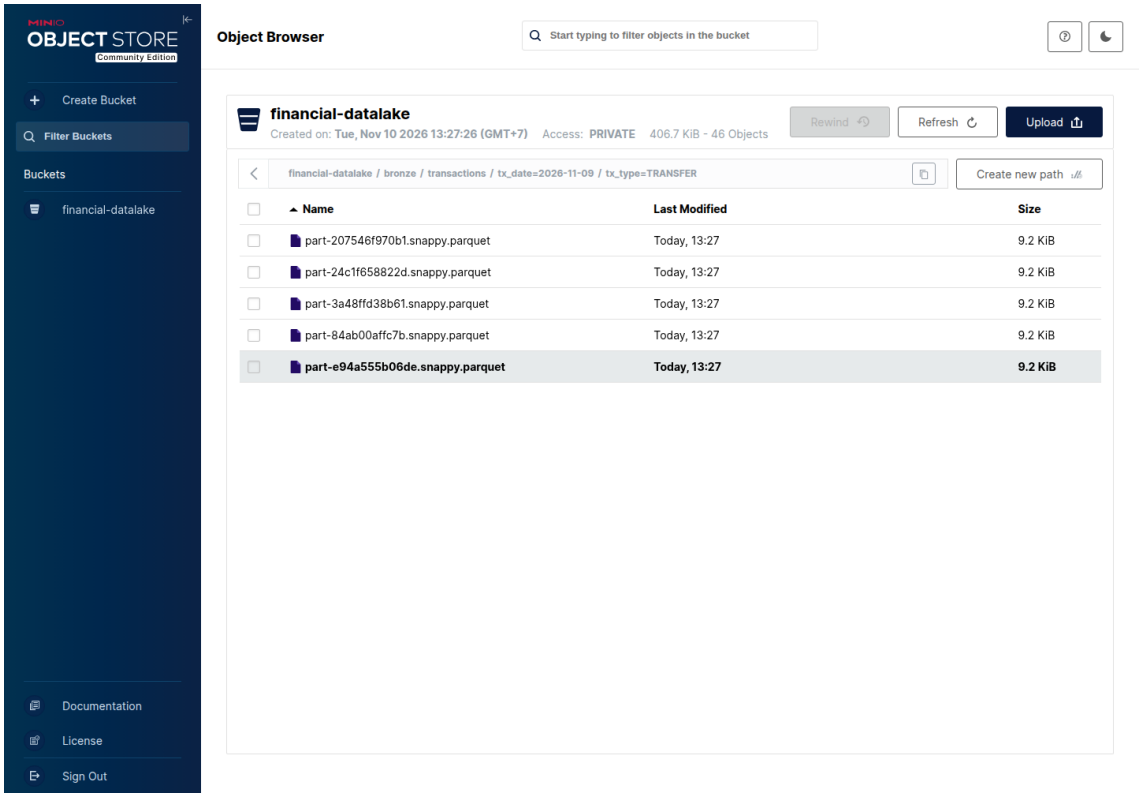
Hình 7.1. Các vùng dữ liệu trong bucket MinIO của thử nghiệm Storage độc lập.



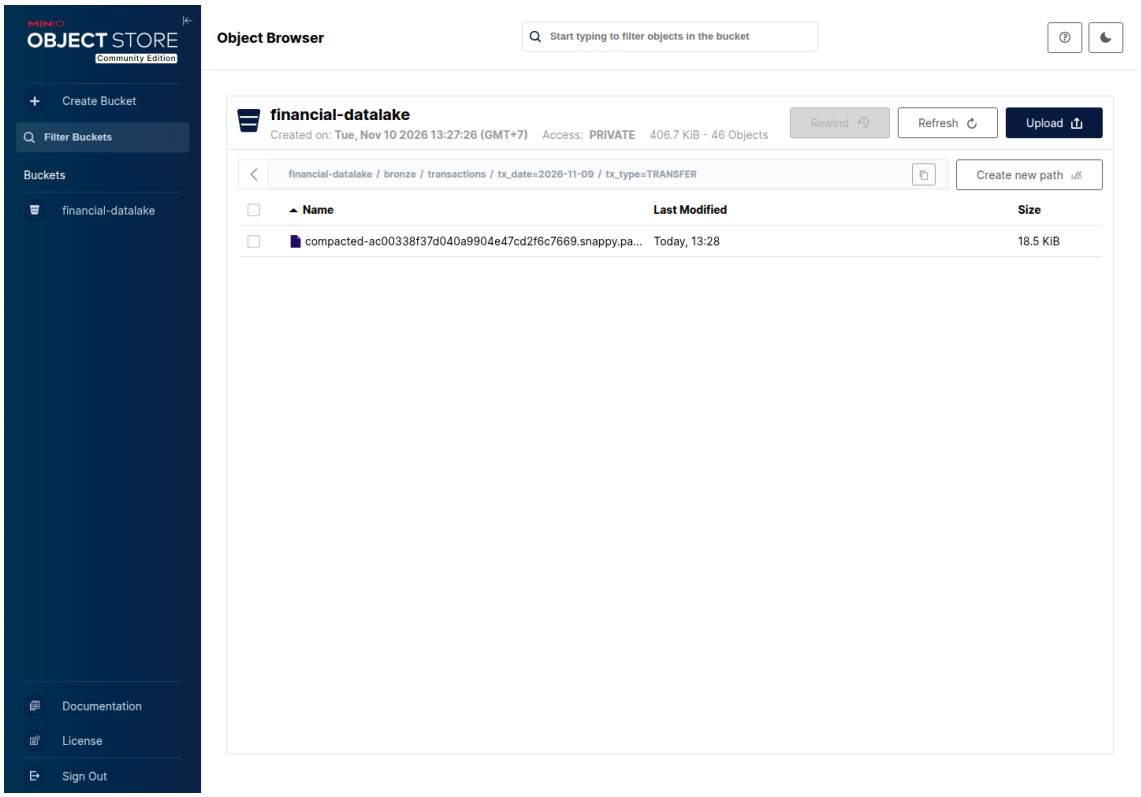
Hình 7.2. Các phân vùng theo ngày giao dịch.



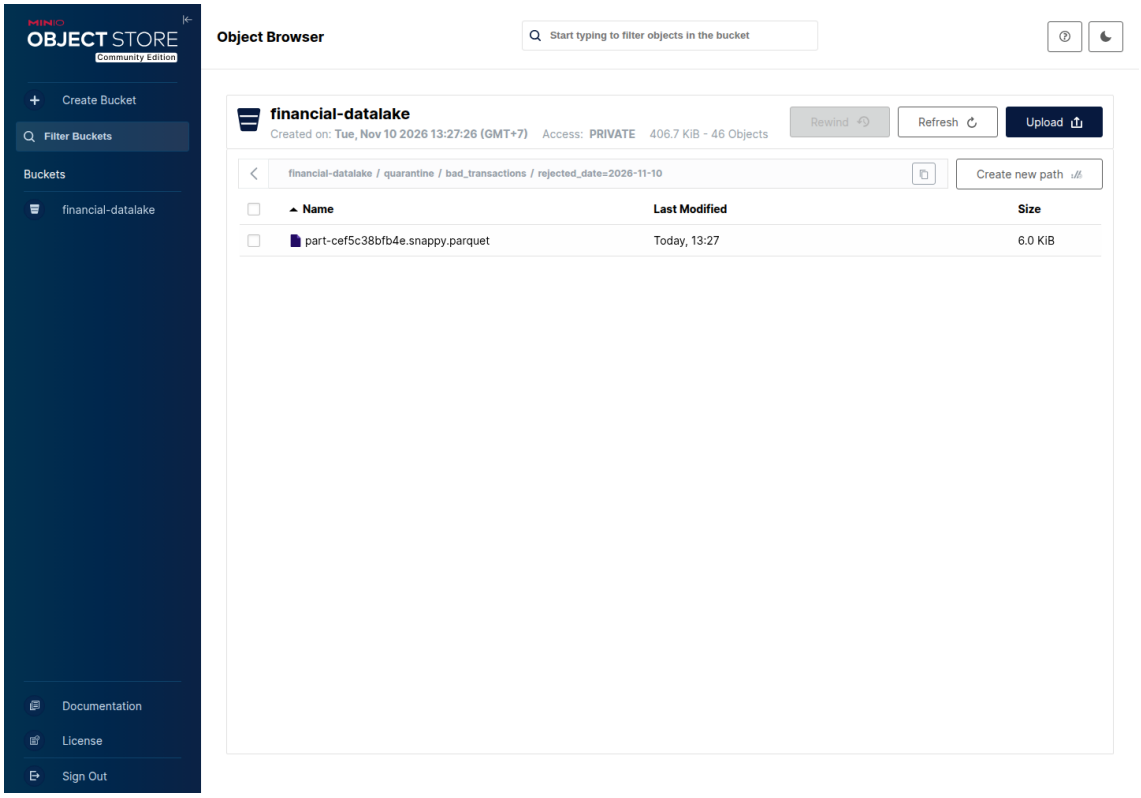
Hình 7.3. Các phân vùng theo loại giao dịch.



Hình 7.4. Các tệp nhỏ trong một phân vùng trước compaction.



Hình 7.5. Tệp sau compaction trong cùng phân vùng.



Hình 7.6. Tệp dữ liệu cách ly trong thử nghiệm Storage.

7.1. Demo toàn trình ngày 12/11/2026

Thử nghiệm dùng môi trường Docker Compose riêng trên một máy: Kafka 3.7.0, MinIO và engine Python microbatch của dự án. Producer sử dụng dữ liệu tổng hợp, mục tiêu phát 3.000 sự kiện gốc, tốc độ đặt 1.000 sự kiện/giây, tỷ lệ chèn lỗi 3% và tỷ lệ tạo bản sao 2%. Các tỷ lệ là xác suất nên số thực tế không nhất thiết đúng tỷ lệ đặt. Topic, consumer group, bucket và cổng mạng được tách riêng khỏi cấu hình mặc định. Không thử nghiệm PySpark trong lượt này.

Chỉ tiêu	Kết quả	Nguồn đối soát
Thông điệp đã gửi và tiêu thụ	3.061	Producer và sink
Giao dịch hợp lệ ghi Bronze	2.923	Producer, sink, Parquet và DuckDB
Giao dịch trùng bị loại	61	Producer và sink
Bản ghi lỗi cách ly	77	Producer, sink và Parquet Quarantine
Message trên DLQ	77	Kafka watermark của topic DLQ
Giao dịch duy nhất trong Bronze	2.923	DuckDB COUNT DISTINCT

Chỉ tiêu	Kết quả	Nguồn đối soát
Bản trùng và không hợp lệ trong truy vấn	0 và 0	DuckDB SQL
Số tệp Parquet Bronze / Quarantine	36 / 12	Liệt kê đối tượng MinIO

Đối soát số lượng: 3.061 = 2.923 + 61 + 77. Sink xử lý trong 3 mẻ. Producer ghi nhận 3,0 giây và 1.020,25 message/giây cho lượt phát; đây là thông lượng producer trong thử nghiệm này, không phải độ trễ hay thông lượng toàn pipeline. Kiểm tra DLQ dựa trên watermark của topic mới có một partition, không phải kiểm tra nội dung từng thông điệp.

Các tệp minh chứng gồm docs/demo_toan_trinh/pipeline_log.txt, result.json và sql_log.txt; bảng CSV truy vấn được lưu tại docs/demo_toan_trinh_sql/. Cấu hình Compose và script thử nghiệm được lưu cùng thư mục minh chứng. Kịch bản này đối soát số lượng và tính duy nhất của giao dịch; chưa xác minh từng payload so với nguồn, chưa ngắt tiến trình để kiểm thử khôi phục và chưa đo hoạt động trên cụm nhiều máy.

Khác với benchmark trong bộ nhớ, demo này thực sự sử dụng producer và sink của dự án qua Kafka và MinIO. Producer ghi log thống kê đầu vào; sink ghi thống kê tiêu thụ và phân nhánh. Bộ kiểm tra độc lập đọc các tệp Parquet bằng PyArrow, đếm transaction_id duy nhất và lấy watermark topic DLQ. Sau đó script của dự án dùng DuckDB đọc trực tiếp S3/MinIO để đếm, kiểm tra trùng, số tiền hợp lệ và xuất bảng tổng hợp [8].

Năm phép đối soát số lượng trong result.json đều đạt: số gửi bằng số đọc; số valid gốc bằng số dòng Bronze; số bản sao khớp số trùng bị loại; số lỗi khớp Quarantine và DLQ; các transaction_id trong Bronze là duy nhất. Phép đếm không chứng minh từng payload không bị thay đổi, và kết quả trong một lần không chứng minh đúng với mọi thứ tự, lỗi mạng hoặc lần khởi động lại.

Kết quả Storage 1.000 bản ghi, 45 tệp thành 9 tệp là một thử nghiệm độc lập ngày 10/11/2026, dùng MinIO cục bộ và fixture tạo sẵn. Không cộng hoặc thay thế các số liệu của thử nghiệm đó với 3.061 message của demo Kafka.

7.2. Trình tự đọc và kiểm chứng minh chứng

Bước đầu đối chiếu docs/demo_toan_trinh/pipeline_log.txt với result.json để xác định lượt chạy: topic demo riêng, 3.061 message đã phát, ba mẻ xử lý và số đếm của producer/sink. Tiếp theo đọc docs/demo_toan_trinh_sql/totals.csv và summary.json để xác nhận 2.923 dòng và 2.923 transaction_id duy nhất, không có bản trùng theo truy vấn hiện hành.

Bảng schema.csv cung cấp kiểu đầu ra; sample.csv minh họa một dòng thực tế; by_partition.csv và by_channel.csv cho thấy dữ liệu có thể truy vấn theo phân vùng và kênh. quarantine.csv ghi tổng hợp lỗi của vùng cách ly. invalid_records = 0 chỉ phản ánh các điều kiện SQL đã cài đặt, không thay thế kiểm tra đầy đủ timestamp, danh mục và kiểu cho toàn bộ schema.

Ảnh MinIO tại Hình 7.1–7.6 thuộc lượt Storage độc lập có 1.000 dòng. Khi trình bày, nhóm giới thiệu chúng cùng docs/tv3_evidence/result.json để giải thích 45 tệp trước và 9 tệp sau compaction. Demo toàn trình 3.061 message được chứng minh bằng bộ log/JSON/CSV riêng; không dùng ảnh của Storage làm ảnh đại diện cho lần chạy Kafka.

Một người kiểm tra có thể đọc các tệp minh chứng đã lưu mà không cần khởi động dịch vụ. Khi muốn xác minh thực thi lại, sử dụng môi trường cô lập và lệnh tại Phụ lục A.4; kết quả của lượt mới phải được lưu riêng cùng thời điểm, cấu hình và mã thoát. Hai cách kiểm tra có giá trị khác nhau: đọc hồ sơ xác nhận tính nhất quán của bằng chứng, còn chạy lại mới xác nhận khả năng tái lập trên môi trường được sử dụng.

CHƯƠNG 8. THẢO LUẬN

Thiết kế tách tiếp nhận, lưu trữ và kiểm tra giúp nhóm quan sát từng thành phần. Parquet và tổ chức theo phân vùng tạo đầu ra thuận tiện cho công đoạn sau. Tuy nhiên cấu hình một broker, MinIO cục bộ và dữ liệu tổng hợp không cho phép đánh giá chịu lỗi cụm, độ trễ mạng hay tải sản xuất.

Khả năng khôi phục microbatch còn hạn chế vì trạng thái chống trùng nằm trong bộ nhớ và thao tác ghi/commit không nguyên tử. Cần kiểm thử tắt tiến trình trước khi ghi, sau khi ghi và trước commit; đối soát theo transaction_id và offset sau khi khởi động lại. Không chạy compaction đồng thời với ghi nếu chưa có cơ chế phối hợp an toàn.

Các hướng mở rộng gồm kiểm thử tải nhiều mức và lặp lại nhiều lần, tăng tính tái lập bằng phiên bản phụ thuộc cố định, triển khai nhiều nút, và khảo sát định dạng bảng hỗ trợ giao dịch. Các hướng này là đề xuất, chưa phải tính năng đã kiểm chứng.

8.1. Đánh giá đặc trưng Big Data và khả năng mở rộng

Velocity được minh họa bằng việc producer gửi message vào Kafka và sink thu theo mẻ. Kafka partition cho phép tổ chức xử lý theo luồng và là cơ sở để phân phối công việc giữa các consumer cùng nhóm, nhưng demo chỉ có một broker và một tiến trình sink nên không đo được mức tăng hiệu năng khi thêm máy [1].

Volume là nhu cầu thiết kế: khi nhiều giao dịch được lưu lâu dài, lưu trữ đối tượng và định dạng cột giúp tổ chức và đọc dữ liệu theo các tập con phù hợp. Kết quả 100.000 bản ghi cho thấy chênh lệch kích thước giữa các định dạng trong phép thử cục bộ, nhưng chưa chứng minh hệ thống đạt khả năng xử lý hàng triệu hay hàng tỷ giao dịch [3], [4].

Veracity được minh họa bằng lỗi cố ý, nhánh cách ly và phép đối soát. Trong demo, 77 bản lỗi được ghi vào Quarantine và 77 message được đếm ở DLQ; đây là kết quả cho ba dạng lỗi được chèn, không bao trùm mọi dạng lỗi có thể xảy ra. Một số trường trong data contract chưa được validator kiểm tra; vấn đề này cần hoàn thiện nếu yêu cầu schema nghiêm ngặt hơn.

Để đánh giá mở rộng, đề xuất các mức tải tăng dần, mỗi mức lặp lại nhiều lần và đo thông lượng đầu-cuối, thời gian lưu theo từng message, CPU, RAM, backlog và số lượng tệp. Sau đó so sánh một và nhiều consumer hoặc nhiều broker trên hạ tầng phù hợp. Đây là kế hoạch thí nghiệm, không phải kết quả đã thực hiện.

8.2. Lập luận về tải và kế hoạch kiểm chứng mở rộng

Ba loại bằng chứng hiện có trả lời ba câu hỏi khác nhau: demo 3.061 message xác nhận các thành phần kết nối và đối soát được số lượng; benchmark 100.000 bản ghi khảo sát xử lý/tệp cục bộ; thử Storage 1.000 dòng kiểm tra bảo toàn dữ liệu khi compaction. Không cộng các bộ này thành một quy mô thử nghiệm chung. Trong benchmark, throughput xử lý trong bộ nhớ tăng khoảng 3,66 lần từ mẻ 1.000 lên 10.000, sau đó gần như không tăng ở mẻ 20.000; đây là cơ sở khảo sát kích thước mẻ, chưa phải bằng chứng mở rộng bằng cách thêm máy.

Nhu cầu Big Data xuất phát từ tích lũy và tốc độ liên tục. Nếu giả định duy trì 1.000 sự kiện/giây trong một ngày thì có 86,4 triệu sự kiện/ngày. Đây là phép tính quy hoạch với giả định tải ổn định, không phải tải đã đo hay cam kết hệ thống chịu được. Kafka tách tốc độ phát khỏi tốc độ ghi và giữ backlog khi sink chậm; Parquet giảm dung lượng trong thử nghiệm cục bộ; phân vùng giúp giới hạn tập dữ liệu cần truy vấn. Các kết quả này hỗ trợ lựa chọn kiến trúc nhưng chưa xác định giới hạn dung lượng hoặc độ bền của triển khai một máy.

Kế hoạch dưới đây là thí nghiệm đề xuất, chưa có số đo. Mỗi mức chạy ít nhất ba lần trong topic, consumer group và bucket riêng; giữ nguyên dữ liệu, quy tắc lỗi/trùng, kích thước mẻ và cấu hình máy để so sánh. Producer và sink cần chạy đồng thời; script demo hiện tại phát xong mới chạy sink nên không dùng trực tiếp để kết luận khả năng bám tải liên tục.

Mức thử đề xuất	Đầu vào	Mục đích
L1	100.000 sự kiện gốc; 500 sự kiện/s	Lấy đường cơ sở
L2	100.000 sự kiện gốc; 1.000 sự kiện/s	So sánh khả năng theo kịp tốc độ phát
L3	100.000 sự kiện gốc; 2.000 sự kiện/s	Tìm dấu hiệu backlog tăng
L4	500.000 sự kiện gốc; 1.000 sự kiện/s	Đánh giá tích lũy dữ liệu, tệp và RAM

Cần ghi số message thực gửi (kể cả bản sao), số hợp lệ, trùng và lỗi; thời gian bắt đầu phát, kết thúc phát và hoàn tất ghi; CPU, RAM, số tệp và tổng byte. Thông lượng xử lý đầu-cuối quy ước = tổng message đã phân loại / thời gian từ bắt đầu phát đến hoàn tất ghi và đối soát. Báo riêng tốc độ ghi Bronze = số dòng hợp lệ / cùng thời gian; không dùng throughput producer thay cho hai chỉ số này.

Đo consumer lag mỗi giây bằng tổng trên các partition của (log-end offset − committed offset) đối với đúng consumer group; ghi giá trị lớn nhất và thời gian đưa lag về 0 sau khi producer dừng. Offset commit chỉ là chỉ báo theo dõi, cần đối soát dữ liệu MinIO sau cùng. Muốn đo độ trễ p50/p95/p99 phải ghi thời điểm phát và thời điểm ghi thành công theo từng transaction_id trên đồng hồ thống nhất; không dùng event_time mô phỏng hoặc ingested_at trước khi ghi thay thế thời điểm hoàn tất.

Một mức tải được coi là kiểm chứng trong phạm vi phép thử khi số gửi bằng số sink tiêu thụ; số hợp lệ/trùng/lỗi khớp đầu ra tương ứng; Bronze không trùng ID; backlog sau khi ngừng phát được xử lý hết trong thời gian quan sát đã công bố. Báo trung bình và độ lệch chuẩn của các lần chạy, đồng thời chỉ rõ các mức không theo kịp. Khảo sát nhiều consumer cần bổ sung cơ chế chống trùng dùng chung hoặc bảo đảm cùng transaction_id về cùng partition; tập ID trong bộ nhớ từng tiến trình hiện tại chưa đủ để kết luận chống trùng toàn hệ thống.

8.3. Thứ tự cải tiến và bài học kỹ thuật

Ưu tiên thứ nhất là thống nhất data contract với validator và schema ghi. Hướng hoàn thiện gồm kiểm tra đủ trường, chuyển kiểu thành giá trị chuẩn, xác thực event_time và số hữu hạn, rồi dùng Arrow schema tường minh. Điều kiện nghiệm thu đề xuất là mọi ca đầu vào đều có nhãn mong đợi, bản lỗi đến đúng Quarantine, bản hợp lệ đọc được theo cùng schema và sink không dừng ngoài dự kiến. Đây là yêu cầu cho lần cải tiến, chưa phải chức năng đã hoàn thành.

Ưu tiên thứ hai là kiểm chứng ghi lặp và khôi phục. Cần dừng tiến trình tại các thời điểm trước ghi, sau ghi và trước commit, sau đó khởi động lại để đối soát ID, offset và nội dung. Việc thêm checkpoint hoặc một lời gọi commit riêng lẻ không tự giải quyết cửa sổ phát sinh trùng; thiết kế lưu trữ và quản lý trạng thái phải được đánh giá cùng nhau.

Ưu tiên thứ ba là chạy producer và sink đồng thời theo các mức tải mục 8.2, ghi lag và thời gian hoàn tất, lặp lại ít nhất ba lần. Chỉ sau khi có các số đo này mới đánh giá khả năng theo kịp luồng liên tục. Nếu tăng số consumer, cần kiểm tra cách phân phối transaction_id và trạng thái chống trùng giữa các tiến trình, tránh suy luận hiệu năng từ số partition đã cấu hình.

Bài học chính của nhóm là thiết kế đầu vào/đầu ra và phép đối soát trước khi diễn giải hiệu năng. Một ảnh bucket có dữ liệu chưa xác nhận đủ số dòng; một benchmark nhanh chưa chứng minh hệ thống chịu được cùng tải qua mạng; một phép thử đếm đúng chưa xác nhận nội dung từng payload. Việc lưu riêng dữ liệu, log và điều kiện của mỗi lượt giúp nhóm trình bày rõ điều đã làm và xác định bước cần kiểm chứng tiếp theo.

CHƯƠNG 9. KẾT LUẬN

Trong phạm vi Ingestion → Storage, nhóm đã xây dựng producer mô phỏng, Kafka, sink microbatch và vùng Parquet phân vùng trên MinIO. Demo đã lưu ngày 12/11/2026 đối soát 3.061 message thành 2.923 giao dịch hợp lệ, 61 bản trùng và 77 bản lỗi. Truy vấn DuckDB xác nhận đọc lại được dữ liệu, số ID duy nhất khớp số dòng Bronze. Thử nghiệm Storage riêng giữ nguyên 1.000 dòng và hash nội dung sau khi gộp 45 tệp thành 9 tệp.

Mục tiêu kết nối các thành phần, tổ chức phân vùng và truy xuất đầu ra đã có bằng chứng trong các kịch bản được mô tả. Mục tiêu cưỡng chế đầy đủ schema, khôi phục sau sự cố và chứng minh khả năng mở rộng nhiều máy chưa được hoàn tất. Đầu ra có thể tích hợp với giai đoạn Storage → Processing khi bên nhận thống nhất schema quan sát được, cách đọc Hive partition, quyền truy cập và giới hạn dữ liệu của lượt bàn giao.

Hướng phát triển tiếp theo là hoàn thiện ràng buộc dữ liệu trước khi ghi, thử nghiệm khôi phục và đo tải liên tục có lặp lại. Các kết quả hiện có tạo cơ sở kỹ thuật cho những bước đó; báo cáo không đưa ra cam kết hiệu năng hoặc độ bền ngoài điều kiện đã kiểm chứng.

TÀI LIỆU THAM KHẢO

[1] Apache Software Foundation, Introduction to Apache Kafka 3.7, https://kafka.apache.org/37/getting-started/introduction/ (truy cập 12/11/2026).

[2] Apache Software Foundation, Structured Streaming Programming Guide, https://spark.apache.org/docs/latest/streaming/index.html (truy cập 12/11/2026). Tài liệu bản mới; nhánh PySpark trong dự án chưa được kiểm thử trong demo.

[3] Apache Software Foundation, Apache Parquet Overview, https://parquet.apache.org/docs/overview/ (truy cập 12/11/2026).

[4] MinIO, MinIO source repository and S3-compatible object storage description, https://github.com/minio/minio (truy cập 12/11/2026).

[5] DuckDB, Reading and Writing Parquet Files, tài liệu phiên bản 2.0, https://duckdb.org/docs/2.0/data/parquet/overview (truy cập 12/11/2026).

[6] Kaggle, Synthetic Financial Datasets for Fraud Detection (PaySim), https://www.kaggle.com/datasets/ealaxi/paysim1 (đường dẫn tham khảo trong README; nội dung trang chưa đọc được tự động).

[7] Đề cương đồ án học phần Nhập môn Dữ liệu lớn, tệp Yêu cầu đồ án.docx.pdf do nhóm cung cấp.

[8] Nhóm thực hiện, mã nguồn và minh chứng nội bộ: README.md, src/, docs/benchmark_results.json, docs/tv3_evidence/, docs/demo_toan_trinh/, docs/demo_toan_trinh_sql/; kiểm tra validator: scripts/check_report_schema.py và docs/schema_validation_probe.json.

PHỤ LỤC A. HƯỚNG DẪN CHẠY VÀ HỒ SƠ BÀN GIAO

A.1. Nguồn mã và điều kiện chuẩn bị

Kho mã được cấu hình tại https://github.com/poww-baoo/bigdata-financial-ingestion-storage. Thư mục src/ chứa producer, ingestion, storage và benchmark; docker-compose.yml là cấu hình hạ tầng mặc định; data/sample_transactions.csv là dữ liệu mẫu; requirements.txt liệt kê thư viện. File PaySim đầy đủ không đi kèm kho; script có thể tự sinh dữ liệu. Nếu dùng kho từ xa cần đối chiếu phiên bản được nộp, vì đường dẫn Git không tự chứng minh mã ở remote đã cập nhật.

Cài Docker Engine, Docker Compose và Python; cần quyền sử dụng Docker và các cổng cấu hình không bị chiếm. Thử nghiệm đã thực hiện dùng Python 3.14.4 và các phiên bản nêu ở mục 6.1; requirements.txt dùng giới hạn tối thiểu chứ chưa khóa toàn bộ phiên bản, vì vậy môi trường khác có thể cho kết quả khác. Nhánh PySpark cần thêm Java và các gói kết nối phù hợp; nhánh này chưa được chứng minh bằng lượt demo trong báo cáo.

A.2. Các lệnh chạy theo cấu hình dự án

```
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
docker compose up -d
docker compose ps
python -m src.producer.transaction_producer --count 3000 --rate 1000 --fault-rate 0.03 --dup-rate 0.02
python -m src.ingestion_sink.spark_kafka_to_lake --engine microbatch --batch-size 1000
python -m src.storage_management.init_minio_buckets
python -m src.storage_management.query_verification
```

Chạy từ thư mục gốc của dự án. Lệnh đầu tạo môi trường Python; lệnh pip cài phụ thuộc. Compose mặc định khởi tạo Kafka, Kafka UI và MinIO theo cấu hình dự án; producer gửi dữ liệu mô phỏng và sink đọc, phân loại, ghi Parquet. Kiểm tra trạng thái dịch vụ trước khi phát dữ liệu. Các lệnh là kịch bản tái lập tham khảo; không tự giả định sẽ tạo lại đúng số 3.061 vì tỷ lệ lỗi/trùng là ngẫu nhiên.

Kafka topic, consumer group, offset và bucket còn dữ liệu từ lần trước có thể làm kết quả đếm khác. Để thí nghiệm không ảnh hưởng dữ liệu cũ, dùng môi trường mới

có tên project, topic, group và bucket riêng hoặc sao lưu, thống nhất điều kiện ban đầu trước khi thao tác. Không xóa volume hoặc dữ liệu cũ chỉ để làm sạch demo. Tài khoản MinIO mặc định trong Compose chỉ dành cho thực hành cục bộ.

Kiểm chứng compaction: dừng writer và các reader liên quan, trước hết chạy python -m src.storage_management.compact_small_files để xem kế hoạch; chỉ dùng tùy chọn --apply sau khi đã xác nhận vùng dữ liệu bảo trì và lưu kết quả đối soát. Không chạy song song với ingestion. Lệnh benchmark python -m src.benchmark.run_benchmarks --records 100000 có ghi lại biểu đồ/kết quả và xóa các thư mục flat_parquet, partitioned_parquet dưới data/benchmark_tmp; cần bảo toàn các kết quả cũ trước khi chạy lại.

A.3. Cấu hình, schema và minh chứng

Cấu hình mặc định gồm KAFKA_BOOTSTRAP_SERVERS, tên topic financial_tx và financial_tx_dlq, endpoint MinIO, bucket, tiền tố Bronze/Quarantine và checkpoint; giá trị được đọc tại src/config.py. Phần dữ liệu hợp lệ lưu theo bronze/transactions/tx_date=.../tx_type=...; phần lỗi lưu dưới quarantine/bad_transactions/rejected_date=.... Các trường và kiểu dự kiến được mô tả tại mục 3.2 và bảng 3.3; không đưa mật khẩu thực tế vào báo cáo hay kho công khai.

Minh chứng của lượt demo ngày 12/11: docs/demo_toan_trinh/pipeline_log.txt, result.json, sql_log.txt; các bảng schema, totals, by_partition, by_channel và quarantine ở docs/demo_toan_trinh_sql/. Cấu hình và script dùng để tạo thử nghiệm cô lập nằm trong docs/demo_toan_trinh/. Script này tạo topic có tên cố định và ghi result ra docs/demo_toan_trinh_result.json; cần dùng môi trường mới, kiểm tra tên topic và sao lưu file kết quả trước khi chạy lại [8].

Bộ minh chứng Storage riêng nằm trong docs/tv3_evidence/ và docs/screenshots/; số liệu benchmark nằm tại docs/benchmark_results.json cùng các hình trong docs/charts/. Khi nộp, gửi mã nguồn/cấu hình, mẫu dữ liệu hoặc script sinh dữ liệu, các log và bảng phân công cùng file PDF theo yêu cầu mục 7 của đề [7].

A.4. Tái lập demo toàn trình bằng môi trường riêng

Các lệnh sau dùng PowerShell tại thư mục gốc dự án, với Docker Engine/Compose v2 và Python đã cài requirements.txt. Cổng 19092, 19000 và 19001 phải còn trống. MinIO dùng thông tin thực hành đã khai báo trong compose.yml; không dùng cho dịch vụ công khai. Lệnh DuckDB cần cài/tải được extension httpfs. Đây là hướng dẫn đối chiếu từ script/cấu hình hiện có; việc chạy lại trên môi trường mới có thể khác phiên bản thư viện và image latest.

Bước 1. Tạo thư mục kết quả mới và khởi động Compose riêng. Không chạy lại demo.py trên topic đã tồn tại: script gọi create_topics với tên cố định và sẽ lỗi. Tên project mới chỉ tách container, không giải phóng các cổng nếu một project cũ vẫn đang chạy. Kiểm tra docker compose ls và trạng thái dịch vụ, giữ nguyên dữ liệu cũ.

```
$run = Get-Date -Format yyyyMMdd_HHmmss
$project = "financial-demo-$run"
$out = "docs/replay_$run"
New-Item -ItemType Directory -Path $out | Out-Null
Copy-Item docs/demo_toan_trinh_result.json "$out/result_previous.json" -
ErrorAction SilentlyContinue
docker compose -p $project -f docs/demo_toan_trinh/compose.yml up -d --
wait
docker compose -p $project -f docs/demo_toan_trinh/compose.yml ps
```

Chỉ tiếp tục khi Kafka và MinIO đều healthy. Compose này không gắn volume dữ liệu được đặt tên; giữ container của lần thử cho đến khi thu đủ minh chứng. Ghi image ID/digest bằng docker compose images để kiểm soát khác biệt do nhãn latest.

Bước 2. Đặt PYTHONPATH để chạy script tại thư mục con, giữ log mới và kiểm tra mã thoát. demo.py tự cấu hình endpoint, topic, group và bucket cho producer/sink; khởi tạo ba partition cho financial_demo_2026 và một partition cho financial_demo_dlq_2026, đặt random.seed(20261112), phát 3.000 sự kiện gốc rồi đọc bằng sink.

```
$env:PYTHONPATH = (Get-Location).Path
```

```
$env:PYTHONUNBUFFERED = "1"
python docs/demo_toan_trinh/demo.py 2>&1 | Tee-Object
"$out/pipeline_log.txt"
if ($LASTEXITCODE -ne 0) { throw "Demo failed; inspect pipeline_log.txt" }
Copy-Item docs/demo_toan_trinh_result.json "$out/result.json"
```

Bước 3. Cấu hình cùng bucket cho tiến trình truy vấn mới. Biến môi trường bên trong demo.py không truyền ngược ra PowerShell, vì vậy cần đặt lại các giá trị dưới đây. Dùng số Bronze vừa đo trong result.json làm expected-records, tránh mặc định mọi lần đều cho đúng 2.923.

```
$env:MINIO_ENDPOINT = "localhost:19000"
$env:MINIO_S3_ENDPOINT = "http://localhost:19000"
$env:MINIO_ACCESS_KEY = "demo_minio_admin"
$env:MINIO_SECRET_KEY = "demo_minio_pass_2026"
$env:MINIO_BUCKET_NAME = "financial-demo-isolated"
$env:LAKE_BRONZE_PATH = "bronze/transactions"
$env:LAKE_QUARANTINE_PATH = "quarantine/bad_transactions"
$r = Get-Content "$out/result.json" -Raw | ConvertFrom-Json
python -m src.storage_management.query_verification --expected-records
$r.bronze_rows --output-dir "$out/sql" 2>&1 | Tee-Object
"$out/sql_log.txt"
if ($LASTEXITCODE -ne 0) { throw "SQL verification failed" }
```

Get-Content "$out/sql/summary.json"

Kết quả mong đợi là năm checks trong result.json đều true; tổng gửi bằng tổng tiêu thụ; valid_sent bằng bronze_rows và bronze_unique_ids; duplicates_sent khớp duplicates_dropped; fault_injected khớp Quarantine và DLQ. SQL xuất schema.csv, totals.csv, sample.csv, by_partition.csv, by_channel.csv, quarantine.csv và summary.json có passed = true, duplicate_records = 0, invalid_records = 0. invalid_records ở đây chỉ kiểm tra ID và amount theo câu SQL, không xác nhận đầy đủ data contract.

Mốc tham khảo của lượt 12/11/2026 là 3.061 message, 2.923 dòng Bronze, 61 bản trùng, 77 lỗi, 36 tệp Bronze và 12 tệp Quarantine. Seed cố định hỗ trợ tái lập dữ liệu ngẫu nhiên, nhưng thời gian, UUID tệp, lịch gom mẻ và phiên bản phụ thuộc có thể làm số tệp hoặc kết quả khác; không sửa log để khớp mốc cũ. Bộ kiểm tra DLQ chỉ đếm watermark trên topic mới, chưa so payload từng message.

Sau khi thu đủ kết quả, có thể dừng dịch vụ bằng: docker compose -p $project -f docs/demo_toan_trinh/compose.yml stop. Không dùng down -v để chuẩn bị lần mới. Muốn chạy một lượt sạch mà giữ dữ liệu cũ, cần đổi đồng bộ cổng trong Compose và endpoint, tên topic, group, bucket trong bản sao script; chỉ đổi tên project không đủ nếu cổng đang được sử dụng.

PHỤ LỤC B. ĐỐI CHIẾU YÊU CẦU VÀ DANH MỤC MINH CHỨNG

B.1. Vị trí đáp ứng yêu cầu bắt buộc

Mục 5.1 của đề được trình bày tại mục 1.1 và 4.1: ranh giới Kafka → MinIO, producer hỗ trợ nguồn và DuckDB kiểm chứng đầu ra. Mục 5.2 được giải thích tại 1.3, 8.1 và 8.2: Velocity, Veracity và nhu cầu tích lũy Volume; các hạn chế về mở rộng được công bố cùng kế hoạch đo.

Mục 5.3 và 5.4 tương ứng Chương 3: nguồn mô phỏng, schema dự kiến, quy tắc đang kiểm tra, ví dụ JSON, đường dẫn đầu ra và một dòng Parquet đã truy vấn. Mục 5.5 và 5.6 tương ứng Chương 2, mục 4.1 và 4.4–4.5: công nghệ, lý do chọn, sơ đồ, giao diện JSON/S3/Parquet và phân vùng.

Mục 5.7 được thể hiện tại Chương 6 và 7 với các bộ chứng cứ có phạm vi riêng. Mục 5.8 nằm ở Chương 5 và Phụ lục A: cấu hình, lệnh chạy, script sinh dữ liệu và hướng dẫn đối soát. Bảng phân công ở đầu báo cáo mô tả trách nhiệm của bốn thành viên; nội dung này là phân công nhóm, không phải thống kê tự động số giờ hoặc số dòng mã.

B.2. Kiểm tra tính đầy đủ của hồ sơ

Hồ sơ bàn giao gồm báo cáo, README.md, requirements.txt, docker-compose.yml, thư mục src/, dữ liệu mẫu data/sample_transactions.csv và script sinh giao dịch. Minh chứng toàn trình gồm docs/demo_toan_trinh/ và docs/demo_toan_trinh_sql/; benchmark gồm docs/benchmark_results.json và docs/charts/; Storage gồm docs/tv3_evidence/ và các ảnh tương ứng tại docs/screenshots/.

Kiểm tra validator được tái lập riêng bằng lệnh dưới đây. Script không kết nối dịch vụ, chỉ trích hàm hiện hành và ghi kết quả kèm hash tệp nguồn. Khi mã thay đổi, cần chạy lại và diễn giải kết quả theo phiên bản mới. Giá trị all_match_documented_behavior = true chỉ có nghĩa hành vi quan sát khớp chín ca mô tả; nhiều ca được chấp nhận vẫn là lỗ hổng so với hợp đồng dự kiến.

```
python3 scripts/check_report_schema.py
```

Trước khi nộp, nhóm cần bảo đảm các tệp được đưa vào gói hoặc phiên bản repository bàn giao. Đường dẫn repository trong báo cáo không thay thế việc kiểm tra các tệp đã được tải lên. Theo mục 7 của đề, bản báo cáo nộp được xuất thành PDF sau khi nhóm duyệt nội dung và cập nhật mục lục; không cần gửi toàn bộ dữ liệu lớn nếu đã có mẫu hoặc script sinh.

PHỤ LỤC C. SỬ DỤNG CÔNG CỤ AI

Trong đợt hoàn thiện báo cáo, công cụ AI được dùng để đối chiếu cấu trúc với yêu cầu học phần, hỗ trợ biên tập nội dung, kiểm tra các tệp minh chứng và tạo script ghi nhận hành vi validator. Công cụ hỗ trợ phân biệt số đo trong bộ nhớ, thử nghiệm Storage và demo Kafka → MinIO, đồng thời rà soát dẫn chiếu và bố cục tài liệu.

Các số liệu demo và benchmark trong bản báo cáo lấy từ hồ sơ đã có của dự án. Đợt biên tập này không thực hiện lại demo Kafka → MinIO hoặc thí nghiệm tải L1–L4; chỉ chạy riêng phép kiểm tra validator và đối chiếu tính nhất quán của minh chứng. Nhóm chịu trách nhiệm đọc lại, kiểm tra nội dung, giải thích mã nguồn và xác nhận báo cáo trước khi nộp.